



Pour les manipulations sur le boîtier, on suppose que vous utilisez 2 logiciels pour le manipuler :

- **Winscp** pour les échanges de fichiers entre le boîtier et le PC
- **Putty** (que l'on peut appeler directement depuis Winscp) pour ouvrir une console en mode SSH sur le boîtier et permettre la compilation et l'exécution des programmes.



1. Utilisation des exemples fournis

A partir de la documentation constructeur (*user_manual*), d'après la section **2.4 Start Programming** :

- Dans quel langage de développement sont fournis les programmes d'exemple ?
- Pour chacun des programmes, précisez le nom du fichier source ainsi que ce qu'il fait.
- A quoi sert le fichier **vsopenrisc.h**.
- D'après la documentation, quelle commande permet de générer les programmes exécutables ?
- Générez ces programmes et testez celui qui concerne les leds et le buzzer.

2. Programmation

Pour la suite, créez un dossier **/home/CIEL2_202X/XXXXXXX/tp2** (où CIEL2_202X est l'année en cours et XXXXXXX est votre nom).

2.1. Premier programme

Créez le programme **tp2-2_1.c** en saisissant le code suivant :

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/ioctl.h>
#include <fcntl.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include "vsopenrisc.h"
int main()
{
    int fd = -1, i;
    unsigned long val;

    fd = open("/dev/gpio", O_RDWR);

    for (i=0;i<3;i++) {
        val=LED_GREEN;
        ioctl(fd, GPIO_CMD_SET_LEDS, &val);
        printf("Allumage led verte\n");
        sleep(1);
    }
}
```



Programmation des boîtiers VSCOM OpenRISC Alena

2- Programmation en C

```
val=0;
ioctl(fd, GPIO_CMD_SET_LEDS, &val);
printf("Extinction led verte\n");
sleep(1);
}

close(fd);

if (fd == -1) return -1; //si on a pas pu ouvrir correctement le port, on sort avec le code -1
return 0;               //sinon le code 0 = ok
} //main
```

Compilez le programme par la commande

```
> g++ tp2-2_1.c
```

Que fait-il ?

Ajoutez des commentaires au programme en précisant les instructions qui :

- Ouvre le périphérique permettant la manipulation des I/O du boîtier
- Effectue l'allumage de la led
- Attend 1 seconde
- Ferme l'accès au périphérique

2.2. Modification du programme

En vous servant du fichier **vsopenrisc.h**, qui définit les constantes liées au fonctionnement hardware du boîtier, modifiez le programme précédent (copié en **tp2-2_2.c**) pour allumer 10 fois la led bleue au lieu de la verte.

2.3. 2ème Modification du programme

Modifier le programme précédent (copié en **tp2-2_3.c**) pour faire bipper le buzzer en même temps que la led s'allume (le programme d'exemple **ioctls.c** peut vous aider dans la commande du buzzer).

2.4. Programmation

Reprenez l'algorithme du programme réalisé dans le précédent TP (allumant successivement les 3 leds (chenillard) défilant chaque seconde, 5 fois de suite, en émettant un bip au début, puis 2 à la fin). Proposez, saisissez (nommé **tp2-2_4.c**) et testez un programme réalisant l'algorithme.

Pour aller plus loin...

- Créez des fonctions pour les parties de codes qui se répètent
 - allumage des leds : fonction `allumeLed(int couleur),`
 - et une pour le buzzer une fonction `bip(int nb, float duree)`
- Prise en compte de paramètres (arguments) en ligne de commande pour modifier le fonctionnement (nombre de cycle, durée...)